

AutoDrive: Turn-Boundary Continuation Control for Long-Horizon Coding Agents

Anonymous Author(s)

Anonymous Institution

Anonymous Country

Abstract

Coding agents often end a conversational turn before the admitted software task is complete. Users then re-enter short prompts such as “continue,” even when the next action is safe, in scope, and inferable from the existing trace. We call this failure *premature session handoff*. We present AUTO DRIVE, a session-scoped controller that evaluates terminal worker turns at safe boundaries, chooses among CONTINUE, STOP, and DEFER, and durably admits bounded follow-up work. The design separates model execution from continuation admission, preserves user queue and steering priority, and uses event-sourced state plus deterministic input identifiers to recover exactly once across a decision-to-admission crash window. We define a preregistered evaluation on all 48 SWE-EVO long-horizon tasks, a 384-trajectory model and policy matrix, a balanced 180-boundary human-labeled corpus, component ablations, and fault injection. Primary outcomes cover task success, partial Fix Rate, manual continuation demand, unsafe continuation, redundant work, cost, latency, and recovery. This manuscript is a pre-results study draft: system and protocol claims are test-backed, while empirical outcome fields remain PENDING until the frozen runs and independent annotations are complete.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Machine learning*.

Keywords

coding agents, autonomous agents, continuation, termination, abstention, long-horizon software engineering, fault tolerance

Draft status. The implementation and v1.14 protocol are available. Four v1.13 paid canaries and one accepted negative v1.14 non-primary pilot are descriptive only. The formal matrix and two-person boundary annotations are incomplete. Every red PENDING field is generated from an explicit result macro and is not an empirical claim.

1 Introduction

Interactive coding agents combine a language model, repository tools, execution feedback, and a conversational interface. Their provider turns are necessarily finite, but the user’s software task may span many such turns. A useful distinction follows: a *worker turn* may end because the model emitted a terminal response, while the *admitted task* remains incomplete. In practice, a worker may summarize partial progress, list unexecuted next steps, or ask whether it should continue. The human then sends a semantically thin message—often only “continue”—that restores execution without adding requirements, authority, or information.

This interaction is not merely inconvenient. It fragments long-horizon execution, makes completion dependent on user vigilance, and confounds evaluation: a capable worker can receive a failing score because its interface yielded control at an arbitrary turn boundary. At the same time, automatically resuming every terminal response is unsafe. A worker may be waiting for a product choice, credentials, permission expansion, destructive confirmation, or missing acceptance criteria. Blind continuation can also create redundant model calls, unbounded loops, cost exhaustion, or repeated external effects.

Prior systems establish important neighboring ideas. ReAct interleaves reasoning and action [14]; coding scaffolds such as SWE-agent and OpenHands provide interfaces and execution environments [12, 13]; and Magentic-One uses an orchestrator to plan, track progress, and recover from errors across specialized agents [3]. Recent work studies sequential abstention [9], evidence-linked completion [8], and infinite agentic loops [5]. These works motivate careful control but do not make a narrower question disappear: at the boundary of a single coding worker’s terminal turn, should the same admitted session continue, stop, or return control to the user?

We study that question through AUTO DRIVE. The controller is scoped to the worker’s repository location and the durable Session. It evaluates only safe turn boundaries and returns one of three actions. CONTINUE admits one bounded queued input; STOP closes the automatic chain; and DEFER returns control when the next action depends on human authority or information. A continuation decision is persisted before queue admission with a deterministic input identifier. Recovery can therefore reconcile a recorded decision with an existing or missing input without duplicating it.

This paper makes four bounded contributions:

- (1) It formulates *premature session handoff* separately from model stopping, task completion, abstention, and infinite loops.
- (2) It describes a three-state, safety-first continuation controller integrated with durable Session admission, user-priority scheduling, bounded retries, and exactly-once crash recovery.
- (3) It provides a preregistered evaluation: 48 SWE-EVO tasks, four policies, three worker models, 384 paid trajectories, a 180-boundary labeled set, cumulative ablations, and nine fault scenarios.
- (4) It releases an evaluation harness that rejects incomplete provenance, enforces a USD 800 budget, derives off-policy prefixes without extra runs, and computes exact McNemar, task-paired bootstrap, Holm-adjusted, kappa, and macro-F1 statistics.

We do not claim that supervisors, memory, termination critics, abstention, or loop bounds are new. The empirical contribution is conditional on the frozen study and will be reported even if AUTO DRIVE produces null or negative utility.

2 Problem Formulation

2.1 Sessions, turns, and boundaries

Let a Session be an admitted goal g , a durable history H_t , a repository location ℓ , a worker policy π_w , and a set of permissions P . A provider turn transforms history and environment state until it emits a terminal finish reason. We call that point a *turn boundary* b_t . Tool-internal continuations needed to settle already emitted calls are part of the same worker turn and are not AutoDrive decisions.

The latent task state at a boundary is one of three operational classes:

$$y_t \in \{C, S, D\},$$

where C means that a safe, concrete, in-scope action remains; S means that the goal is complete or no useful continuation is warranted; and D means that continuing requires a subjective choice, missing information, expanded authority, or a dangerous/external action. A boundary controller π_c predicts

$$\hat{y}_t = \pi_c(g, H_t, \ell, P, m_t),$$

where m_t is optional within-session controller memory. We call a boundary a *premature session handoff* when $y_t = C$ but the interface yields to the user rather than admitting an equivalent safe continuation.

2.2 Safety asymmetry

The error costs are asymmetric. Predicting C for S wastes work and can cause repeated effects; predicting C for D can bypass human control. Predicting S or D for C preserves safety but forfeits autonomy. Therefore, AUTO DRIVE gives DEFER precedence whenever authority or information is uncertain, and bounds each automatic chain even when all decisions are CONTINUE.

We measure task utility and boundary classification separately. A controller can classify boundaries accurately yet fail to improve task resolution because the worker cannot solve the remaining problem. Conversely, blind continuation can improve resolution while producing unacceptable unsafe or redundant turns. No single success rate captures both dimensions.

2.3 Research questions

The study asks:

- RQ1** How often do long-horizon coding trajectories contain premature handoff boundaries?
- RQ2** How does continuation control affect resolution, partial progress, and manual continuation demand?
- RQ3** Which controller inputs—supervision, initial goal, trajectory summary, and within-session memory—contribute to boundary decisions?
- RQ4** What safety, redundancy, token, cost, latency, and recovery trade-offs follow?

3 System Design

3.1 Design overview

Figure 1 shows the control path. The worker owns model/tool execution. The controller runs only after the worker and pending user work reach a safe boundary. Its decision is not an in-memory loop flag; it becomes durable Session state. A CONTINUE decision carries a chain identifier, continuation number, deterministic input identifier, reason, and next prompt. The input is then admitted using the same queue abstraction as human follow-ups.

3.2 Location and Session scope

Repository tools, permissions, model resolution, and filesystem context are location-scoped. Accordingly, the controller is resolved from the same Location service graph as the Session runner rather than from a process-global policy singleton. Its settings are Session state with the following precedence: Session override, project configuration, then built-in defaults. The built-in state is disabled, supervisor policy, at most five continuations, contextual prompting off, and within-session database memory on. A legacy boolean supervisor setting maps to the new policy without silently changing existing configurations.

The public V2 endpoint updates the Session state used by the UI, settings dialog, and slash command. A legacy V1 server rejects the operation rather than presenting a local toggle that did not change runtime behavior.

3.3 Three-state decision

The deterministic policy examines the trailing worker response for completion, continuation, choice, missing-information, permission-expansion, and destructive-action cues. Safety cues produce DEFER before continuation patterns are considered. The supervisor policy receives the admitted initial goal, last worker output, optional trajectory context, and optional within-session memory. It must return strict JSON with an action, reason, optional next prompt, and optional memory update. Its tool set is empty, temperature is zero, output is capped at 1,024 tokens, and the call has a 15-second timeout.

Malformed output, provider failure, an unavailable model or Session, an empty response, or timeout produces DEFER under protocol v1.14. The controller therefore abstains instead of converting uncertainty into a regex guess or an automatic continuation. Traces record the failure path separately from a semantic supervisor decision. Project playbooks are read only when explicitly configured. The default memory lives in the Session database and never writes `.opencode/auto-drive.md` into the worktree.

3.4 Safety policy and action semantics

The controller does not grant authority. It evaluates whether another worker turn may proceed under the authority already attached to the Session. Table 1 makes this distinction operational. A next step can be technically feasible yet still require DEFER because it changes product behavior, spends money, publishes data, messages a person, deploys to a shared environment, or crosses a

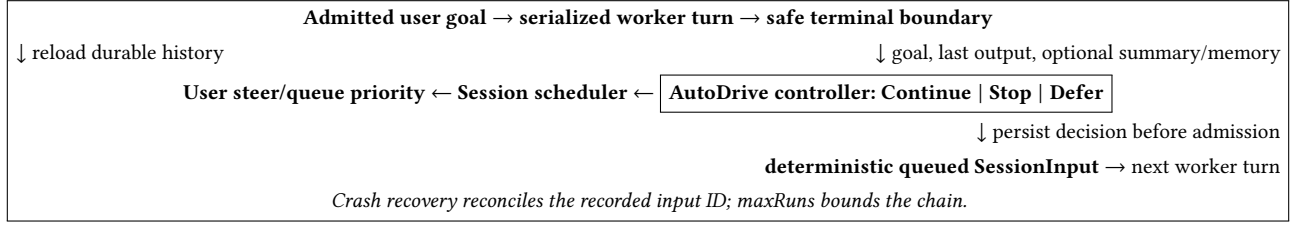


Figure 1: AutoDrive controls Session continuation at worker turn boundaries. Durable admission remains separate from model execution.

Table 1: Decision semantics. Defer takes precedence on uncertainty about authority or information.

Action	Necessary conditions and effect
Continue	Goal incomplete; concrete, in-scope, safe next step; admit one bounded queued input.
Stop	Completion is supported or no useful in-scope action remains; close the chain.
Defer	Choice, information, permission, or safety is unresolved; return control without admission.

destructive boundary. Conversely, a read-only diagnostic can remain CONTINUE when it is a routine way to resolve uncertainty inside the admitted scope.

Let $A(b)$ denote the set of concrete next actions visible at boundary b , $\text{scope}(a, P)$ whether action a is within the current permission set, and $\text{safe}(a)$ whether it avoids a destructive or externally consequential transition. The deterministic admissibility condition is

$$\text{admit}(b) = \neg \text{complete}(b) \wedge \exists a \in A(b) : \text{scope}(a, P) \wedge \text{safe}(a) \wedge \neg \text{choice}(a).$$

This predicate is not presented as a complete semantic model of software work. It documents the asymmetry implemented by the deterministic baseline and the human rubric: evidence of missing authority dominates evidence that work remains. The supervisor can supply a better interpretation of the trace, but cannot weaken this interface contract; failure to obtain that interpretation is itself a reason to defer.

3.5 Durable exactly-once admission

The critical crash window lies between deciding to continue and admitting its follow-up. AUTO DRIVE closes it with the sequence below:

- (1) Choose a deterministic input ID and persist an auto-drive. decided event containing the chain and continuation number.
- (2) Admit a queued SessionInput with that exact ID and source metadata.
- (3) On every drain start, reconcile any persisted CONTINUE decision with the input store.
- (4) If the input is absent, admit it. If an equivalent input exists, reuse it. If the same ID conflicts in Session, prompt, delivery, or source, fail closed.

This is exactly-once admission, not exactly-once provider execution. Provider work after a crash has no durable run identity in the current process-local scheduler; the paper does not claim clustered execution recovery. Duplicate advisory wakes coalesce at the Session coordinator, and durable inputs remain the source of truth.

Recovery is a reconciliation operation rather than a retry loop. For a persisted continuation decision $d = (s, c, n, i, p)$ with Session s , chain c , sequence number n , input ID i , and prompt p , the reconciler performs one of three transitions. If i is absent, it admits (i, p) and records the source metadata. If i names an equivalent pending input, it returns that row without mutation. If i names a non-equivalent input, it raises a conflict and admits nothing. Thus a crash can delay admission, but it cannot select a second ID or increment the chain twice.

The two persistence records serve different audit purposes. The updated event records settings changes and their effective source. The decided event records the controller action, reason, chain, count, prompt identity, policy path, and optional memory transition. The queued input repeats the chain and decision metadata so that transcript consumers can attribute an automatic user-like message without changing the schema of genuine user input.

3.6 Scheduling and bounds

Real user steering inputs are promoted before queued inputs. A user message begins a new automatic chain rather than inheriting a previous continuation count. Automatic follow-ups use queue delivery, so they do not interrupt an unsafe active point. Once the current drain would otherwise become idle, one queued input is promoted and the continuation condition is reevaluated.

The persisted counter prevents a restart from resetting the limit. A request for continuation number $k + 1$ becomes STOP when k already equals maxRuns . Provider retry uses a finite policy; indefinite waiting is prohibited. Permission denial or user rejection interrupts the active ownership chain and leaves unrelated durable inputs intact.

4 Evaluation Methodology

4.1 Primary benchmark

SWE-bench established repository-level issue resolution as an agent benchmark [6]. SWE-EVO extends the setting to release-level software evolution. Its current public version contains 48 tasks from seven Python repositories, with an average patch spanning 21 files

Table 2: Frozen end-to-end trajectory matrix.

Worker setting	Tasks	Repeats	Trajectories
DeepSeek Pro primary	48	1	192
DeepSeek Pro subset repeats	12	2 extra	96
Qwen Max replication	12	1	48
DeepSeek Flash replication	12	1	48
Total			384

and validation averaging 874 tests [7]. We pin all 48 tasks to upstream commit 9b83d5af and Arrow SHA-256 prefix 74e7c631; the artifact records both full digests.

We do not use SWE-Bench Pro for the primary conclusion. An official audit estimated that roughly 30% of its public tasks are broken and identified overly strict tests, underspecified prompts, low coverage, and misleading prompts [11]. This choice does not imply that SWE-EVO is flawless; Section 10 treats benchmark validity as a separate threat.

4.2 Policies and run matrix

Table 2 freezes the paid trajectory count. The four policies are: an oracle that uses an external task verifier to inject a static continuation only while incomplete; blind continuation to the bound; the deterministic regex policy; and the full supervisor policy. The off condition is extracted from the first-boundary prefix of every paid trace and never triggers another model call.

The primary worker is d-robotics/deepseek-v4-pro. The twelve-task subset receives two extra primary-worker repeats and one run each with d-robotics/qwen3.7-max and d-robotics/deepseek-v4-flash. Every supervisor is d-robotics/qwen3.8-max, isolating one source of controller variation. Qwen provides a different-family replication, whereas DeepSeek Flash is only a same-family scale or variant replication. The subset covers every repository and samples low/high upstream PR and FAIL_TO_PASS proxies. Gateway model aliases are recorded as such and are not represented as direct-provider revisions.

Each run uses a pinned image digest and base commit, temperature zero, six worker steps per segment, five automatic continuations, 45 minutes, and at most two concurrent tasks. Exact provider model versions and normalized request hashes are mandatory trajectory fields. The host executor holds provider credentials; task containers do not receive them.

Before either corpus collection or formal dispatch, a separately budgeted non-primary pilot must pass on SWE-bench Verified task psf__requests-1142. The pilot pins dataset revision c104f840, the Parquet and task-input hashes, the official harness commit, and the public AMD64 image digest. It writes only to an isolated pilot index and never contributes to an RQ estimate. One v1.14 run was accepted as a negative pilot: it remained unresolved with F2P 0/1, P2P 5/5, Fix Rate zero, no automatic continuation, 20,014 total tokens, and USD 0.121062 cost. Formal dispatch remains separately gated.

4.3 Execution and isolation

The harness treats the host executor as a narrow trust boundary. Before dispatch it reserves the run’s maximum category charge and passes a frozen run object containing the task, worker, controller, strategy, repeat, timeout, and expected repository provenance. The executor creates the pinned task container without provider credentials. Model requests leave through the host; repository commands run inside the task container. The returned record is accepted only when its run and attempt identities match the reservation and all mandatory provenance fields parse.

A trace preserves every provider request and response, tool invocation, boundary snapshot, controller decision, queue admission, promotion, verifier call, and cost event. Before the worker prompt, the executor verifies the frozen HEAD and clean tracked state, then records pre-existing untracked files as a content-addressed manifest, binary patch, and Git tree. Boundary and final patches are computed relative to that tree. Updates below startup-only roots are omitted from the model patch but retained as trace provenance, so a charged trajectory remains graded and counted. Trajectory schema v4 makes this baseline evidence mandatory for boundary-source and formal acceptance. Raw traces are content-addressed, while the public index contains only their paths and hashes. This separation supports independent integrity checks without placing credentials or unrelated host state into the artifact. Secret-shaped strings cause rejection before append. The task container is discarded after grading, so memory and filesystem changes cannot cross task identities.

The oracle policy does not inspect a hidden reference patch. It invokes the same external task verifier used for final grading at a worker boundary and injects the frozen continuation string only when that verifier reports incomplete. Oracle therefore estimates the opportunity created by better boundary timing under an expensive verifier; it is not an upper bound on worker capability. Blind, regex, and supervisor policies do not receive intermediate test labels from this verifier.

4.4 Boundary corpus

The source sampling frame is independent of the 384 formal rows: 96 supervisor-only trajectories, comprising all 48 SWE-EVO tasks with two repeats. A dedicated source marker in the deterministic run key prevents identity overlap. A boundary-scope capacity receipt, at most two concurrent tasks, a USD 102 category cap, and an external boundary/ index are mandatory. Source outcomes cannot enter RQ2 or the formal cost ledger.

From this frame we sample 180 real worker boundaries, balanced as 60 per class. Base trajectories are grouped into 54 development and 126 frozen test items. Extraction first verifies the trajectory, request, trace, model-metadata, preflight, and captured-patch hashes; it then removes hidden reasoning and the supervisor’s observed decision. Two independent annotators receive a blinded packet and identity-bound CSV. They provide a class, evidence sentence, and next action or missing input; a distinct adjudicator resolves disagreements only after both files are sealed. The corpus freezes only after Cohen’s $\kappa \geq 0.75$ [1]. Prompt or regex tuning on frozen test labels is prohibited.

The cumulative ablation sequence is: regex; supervisor with last output only; plus initial goal; plus trajectory summary; plus within-session memory. This order separates model supervision from progressively richer state. The primary boundary metric is macro-F1, with classwise F1 and STOP/DEFER unsafe-continuation rates.

Boundary sampling occurs after source trajectories are collected and before controller evaluation or formal dispatch. A boundary may enter the pool only when it has a complete admitted goal, the terminal worker response, a repository-state summary, permission state, and a stable base-trajectory identifier. Sampling is stratified iteratively until each target class contains 60 independently agreed examples; class balancing is reported and is not used to estimate population frequency. RQ1 therefore uses the unbalanced first-boundary sample from all accepted source trajectories, including trajectories with no usable later boundary, whereas RQ3 uses the balanced classification set. Conflating these two estimands would bias the reported handoff frequency toward one third.

4.5 Fault injection

Nine scenarios target both correctness and safety: crash after decision/before admission; crash after admission/before promotion; duplicate wake; user input preemption; supervisor timeout; invalid supervisor JSON; retryable provider error; non-retryable provider error; and maximum continuation count. An accepted recovery must preserve input identity, queue priority, chain counter, and trace classification. Infrastructure failures alone may be rerun once with identical parameters, and only when no provider charge occurred.

4.6 Measures and analysis

Resolved rate is the official binary task outcome. Fix Rate follows SWE-EVO’s partial metric over FAIL_TO_PASS and PASS_TO_PASS tests [7]. We additionally measure manual continuations, redundant turns, prompt/completion tokens, cost, wall latency, and recovery. A model timeout, loop, or exhausted budget counts as failure rather than missing data.

A manual continuation is a user-originated input whose normalized content requests progress without adding requirements, permission, or information. Automatic inputs never increment this count. A redundant turn is one in which the worker neither changes the repository, increases Fix Rate, produces new diagnostic evidence, nor resolves a previously documented uncertainty. Two reviewers inspect ambiguous redundancy cases after quantitative outcomes are frozen. Unsafe continuation is counted whenever the controller admits after a frozen STOP or DEFER label; the two source labels remain separate in reporting.

Paired binary outcomes use a two-sided exact McNemar test [10]. If b tasks resolve only under the baseline and c only under treatment, the two-sided exact value is $2\Pr[X \leq \min(b, c)]$ for $X \sim \text{Binomial}(b + c, 0.5)$, capped at one. Concordant tasks do not enter this test.

Task-level differences use 10,000 paired bootstrap resamples with percentile 95% intervals [2]. The resampling unit is the benchmark task, preserving within-task policy pairing and combining repeats inside the task before resampling. A fixed analysis seed is part of

Table 3: Primary SWE-EVO outcomes. Values remain pending until all accepted traces pass provenance checks.

Policy	Resolved	Fix Rate	Manual prompts
Off prefix	PENDING	PENDING	PENDING
Oracle	PENDING	PENDING	PENDING
Blind	PENDING	PENDING	PENDING
Regex	PENDING	PENDING	PENDING
Supervisor	PENDING	PENDING	PENDING

the environment record. Preregistered policy families use Holm correction [4]; raw and adjusted values are both released. The analysis harness tests these implementations against known cases and derives every numerical manuscript macro from validated JSONL. No failed trajectory is converted to missing, and no post-hoc exclusion changes the primary frozen table.

For RQ2, the primary estimand is the task-averaged supervisor-minus-off difference on the 48-task primary run. Regex, blind, and oracle contrasts are secondary members of the corrected family. For RQ3, the estimand is macro-F1 on the 126 frozen boundary items, accompanied by classwise confusion counts. For RQ4, safety rates use label-conditioned denominators, while cost and latency use all dispatched runs. Cross-model estimates retain the same definitions but are explicitly descriptive because the twelve-task subset is not powered as a replacement primary sample.

4.7 Budget and artifact gates

The hard cap is USD 800: 50 pilot, 360 primary, 288 cross-model, and 102 boundary/recovery. Before dispatch, the runner reserves no more than USD 1.25 per primary trace, USD 3.00 per cross-model trace, or USD 102/96 per boundary-source trace. It stops before a reservation could exceed a category or total cap. Each accepted trajectory includes the resolved model, request hash, image digest, code commit, trace hash, tokens, and cost. Secret-shaped material fails validation.

5 RQ1: Premature Handoff Frequency

The primary estimate is the fraction of first boundaries whose frozen human label is CONTINUE. We also report boundaries per trajectory, task-level clustering, and frequency by repository and worker. The preregistered answer is PENDING. No frequency is inferred from regex matches or from the existence of a “next steps” phrase; only frozen labels count.

We will examine whether handoffs concentrate near agent step limits, after summaries, after partial test execution, or after provider stop reasons. This analysis distinguishes interface termination from substantive inability. It is descriptive and does not reclassify benchmark outcomes.

6 RQ2: Task Utility

Table 3 is generated from the frozen trajectory index. The main paired contrast is full supervisor versus each trace’s first-boundary prefix, followed by supervisor versus regex and blind policies. Oracle estimates the continuation opportunity under the same cap, not an achievable autonomous controller.

Before the formal matrix, we completed a paid v1.13 execution canary on one DVC task. Table 4 reports all four accepted policy trajectories. The canary used the same D-Robotics primary worker and controller identifiers as the v1.14 formal protocol, but its historical v1.13 run IDs are not formal matrix rows. Every trajectory was unresolved at its own first boundary; oracle and blind ultimately resolved, while regex stopped immediately and the supervisor resumed once before a controller timeout triggered the then-frozen regex fallback. These rows are independent, unpaired trajectories, so they establish mechanism and failure cases rather than a treatment effect.

The supervisor’s resolved-rate difference is PENDING. We will not interpret a non-significant result as equivalence. A controller may reduce manual prompts without changing resolution, or raise Fix Rate without reaching full resolution. These outcomes are reported independently.

7 RQ3: Component Contributions

The frozen boundary macro-F1 for the complete system is PENDING. The cumulative ablation table will show whether the initial goal resolves ambiguous “done” statements, whether a trajectory summary improves recognition of remaining work, and whether memory prevents repeated failed strategies. We also report per-class confusion matrices because macro-F1 can hide unsafe directionality.

Memory is evaluated within Session only. No default worktree file is written, and no information transfers across benchmark tasks. Thus an observed memory effect represents continuity inside an admitted task, not cross-task training or retrieval.

8 RQ4: Safety, Cost, and Recovery

The STOP unsafe-continuation rate is PENDING and the DEFER unsafe-continuation rate is PENDING. We report them separately because redundant work after STOP and autonomy violations after DEFER have different operational costs. Blind continuation is expected to define a high-autonomy reference, not a safe deployment policy.

The supervisor’s token/cost difference is PENDING and latency difference is PENDING. Recovery succeeds in PENDING of injected cases under the preregistered invariant. These placeholders will be populated only by the analysis pipeline; fault unit tests establish mechanism correctness but do not substitute for end-to-end recovery measurements.

9 Error Analysis

After the frozen run, two reviewers will assign one primary failure category and optional secondary categories to unresolved or unsafe cases. Planned categories are: worker capability failure; wrong continuation decision; correct continuation with ineffective next prompt; repeated strategy; stale or misleading summary; memory contamination within a Session; verifier mismatch; permission/defer error; provider/runtime failure; and benchmark defect.

The v1.13 canary already supplies two observed cases without changing the frozen taxonomy. Regex produced a missed continuation: it persisted STOP on an empty patch even though the worker listed unfinished implementation and validation and the F2P test failed. The full supervisor correctly recovered that boundary and

durably admitted one queue continuation. At the next boundary, its eventual semantic action was again CONTINUE, but the streamed response completed 16.839 seconds after release and missed the 15-second deadline; the v1.13 regex fallback then persisted STOP. The extra segment produced no patch and is counted as redundant. This is classified as a provider/runtime latency failure with a secondary wrong fallback decision, not as evidence that supervision generally harms utility. Before any formal row was accepted, v1.14 changed failure handling to DEFER while preserving the 15-second deadline and all experimental factors.

The accepted v1.14 non-primary pilot exposes the corresponding abstention path. Its controller produced valid CONTINUE JSON 30.241 seconds after request release, but the 15-second deadline had already persisted DEFER. No continuation occurred, the task remained unresolved, and all seven provider responses plus usage and settled cost were retained. We classify this as provider/runtime latency rather than a semantic policy error. It is a single descriptive pilot, not an ablation row or evidence of treatment effect.

We will sample both false continuations and missed continuations, preserving task and worker strata. Error categories are descriptive and cannot remove a failure from quantitative analysis. Suspected benchmark defects are reported with evidence and sensitivity analyses, but the primary frozen result remains intact.

10 Threats to Validity

Construct validity. A static “continue” input is only an approximation to human follow-up. Manual continuation count measures interaction demand, not user satisfaction. The three labels compress nuanced states; the guide and dual annotation mitigate but do not eliminate disagreement.

Internal validity. Provider services may be nondeterministic at temperature zero, and model aliases may move. We therefore record resolved model versions and request hashes, repeat the primary worker on a stratified subset, and pair each off prefix with its own trace. The same Qwen 3.8 Max controller across workers isolates one source of variation but may favor outputs it understands better. The gateway aliases do not reveal direct upstream revisions, limiting reproducibility outside the sealed gateway snapshot.

The single-task canary is especially vulnerable to this threat: its policy rows are independent trajectories, not counterfactual replays. We therefore report its costs, continuations, and outcomes descriptively and do not include it in McNemar tests, bootstrap intervals, or Holm correction.

The non-primary task image also contained untracked build/ before worker execution. Consequently, the pilot’s identical first and final build-only patch hashes preserve repository provenance but their byte and file counts cannot be attributed to the model. The official tests and unresolved classification remain usable. The schema-v4 baseline gate prevents recurrence by computing model patches from the recorded startup tree. Quarantining startup-root changes could hide intentional edits to generated artifacts, so every excluded path remains in the trace and must be reported in artifact diagnostics.

Benchmark validity. SWE-EVO is public, Python-only, and uneven across repositories; DVC contributes many tasks. Public histories may be contaminated in model training. The benchmark’s

Table 4: Single-task v1.13 paid canary ablation on `iterative__dvc_2.21.1_2.21.2`. The four rows are independent, unpaired trajectories and are descriptive only. First and Final are binary resolution outcomes; Gain is the Fix Rate change from the trajectory’s own first boundary.

Policy	First	Final	Gain	Cont.	Redun.	Worker	Ctrl.	Tokens	USD	Sec.
Oracle	0	1	1	5	4	36	0	505675	0.5665	1173.1
Blind	0	1	1	5	4	34	0	438262	0.4757	1040.6
Regex	0	0	0	0	0	6	0	23889	0.0484	436.1
Supervisor	0	0	0	1	1	12	2	96177	0.2837	532.9

tests can still be incomplete or implementation-specific. We publish task-level results and do not generalize to all software development.

External validity. The system is integrated into one agent runtime with durable Sessions. Agents without equivalent queue, event, or location abstractions may require different recovery mechanisms. The study covers coding tasks, not web purchase, messaging, trading, deployment, or other high-impact domains where DEFER policies should be stricter.

Statistical conclusion validity. Forty-eight tasks limit power, and the twelve-task cross-model set is descriptive. We report intervals and corrected tests rather than using thresholded significance as the only evidence. Results are completed and released even if effects are null or negative.

11 Related Work

Coding agents and benchmarks. SWE-bench evaluates issue resolution in real repositories [6]. SWE-agent shows that the agent-computer interface materially affects coding performance [13], while OpenHands provides a general platform with sandboxed execution and benchmark integration [12]. SWE-EVO expands the horizon from single issues to release-level changes and introduces partial Fix Rate [7]. AUTO DRIVE changes neither worker capability nor task tests; it controls whether an admitted Session gets another bounded worker turn.

Orchestration and reflection. ReAct alternates reasoning with environment action [14]. Magentic-One’s lead orchestrator plans and re-plans across a multi-agent team [3]. AUTO DRIVE is narrower: a Location-scoped controller observes a single worker only at terminal turn boundaries and emits a durable Session scheduling decision. We therefore do not claim the first supervisor or progress tracker.

Stopping and abstention. Agentic Abstention frames stopping under uncertainty as a sequential decision and shows that timing matters [9]. Evidence-Carrying Termination gates COMPLETE on typed, replayable evidence [8]. These works emphasize avoiding unsupported action or completion. AutoDrive’s three states incorporate both concerns, but its central target is the complementary failure: returning control while safe work remains. It does not certify external truth or completion evidence.

Loop safety. Infinite Agentic Loops arise from unbounded feedback paths across model, tool, framework, and runtime behavior [5]. AutoDrive uses a persisted continuation limit, finite provider retries, and a DEFER state. These are runtime bounds, not a static detector and not a proof that all loops are absent.

12 Conclusion

Provider turn completion and software task completion are different events. AUTO DRIVE makes that distinction explicit with a three-state boundary controller and durable, bounded, exactly-once follow-up admission. The implementation unifies API, UI, and slash-command control; preserves human priority; avoids implicit work-tree memory; and fails closed on conflicting recovery state. A frozen evaluation will determine whether these mechanisms improve long-horizon coding outcomes at acceptable safety and cost. Until those runs and independent annotations complete, this paper claims a tested system and a reproducible protocol—not an empirical gain.

References

- [1] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- [2] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, New York, NY.
- [3] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. *Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks*. Technical Report MSR-TR-2024-47. Microsoft Research, Redmond, WA. <https://arxiv.org/abs/2411.04468>
- [4] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.
- [5] Xinyi Hou, Shenao Wang, Yanjie Zhao, and Haoyu Wang. 2026. When Agents Do Not Stop: Uncovering Infinite Agentic Loops in LLM Agents. <https://arxiv.org/abs/2607.01641> arXiv:2607.01641.
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? <https://arxiv.org/abs/2310.06770> International Conference on Learning Representations; arXiv:2310.06770.
- [7] Tue Le, Minh V. T. Thai, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. 2025. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. <https://arxiv.org/abs/2512.18470> arXiv:2512.18470.
- [8] Jason Liu. 2026. When May an Agent Stop? Evidence-Carrying Termination for Tool-Using LLMs. <https://arxiv.org/abs/2608.23623> arXiv:2608.23623.
- [9] Han Luo, Bingbing Wen, and Lucy Lu Wang. 2026. Agentic Abstention: Do Agents Know When to Stop Instead of Act? <https://arxiv.org/abs/2606.28733> arXiv:2606.28733.
- [10] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- [11] OpenAI. 2026. Separating Signal from Noise in Coding Evaluations. <https://openai.com/index/separating-signal-from-noise-coding-evaluations/> Accessed 2026-08-30.
- [12] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. <https://arxiv.org/abs/2407.16741> arXiv:2407.16741.
- [13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. <https://arxiv.org/abs/2405.15793> Advances in Neural Information Processing Systems; arXiv:2405.15793.

- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. https://openreview.net/forum?id=WE_vluYUL-X International Conference on Learning Representations; arXiv:2210.03629.

A Frozen Protocol Details

A.1 Twelve-task replication subset

The subset is fixed before outcomes are observed:

- Conan: 2.0.2-2.0.3, 2.0.14-2.0.15;
- Dask: 2023.9.2-2023.9.3, 2024.1.0-2024.1.1;
- DVC: 0.30.0-0.30.1, 3.13.3-3.14.0, 2.8.1-2.8.2;
- Modin: 0.24.0-0.24.1, 0.25.0-0.25.1;
- Requests: v2.4.0-v2.4.1;
- Pydantic: v2.7.0-v2.7.1;
- scikit-learn: 0.21.1-0.21.2.

The selection covers all seven repositories and intentionally includes contrasting upstream PR and FAIL_TO_PASS counts. It is not optimized against AutoDrive outcomes.

A.2 Dataset composition

Table 5 summarizes the frozen manifest before any outcome is observed. Counts are descriptive properties copied from the pinned SWE-EVO Arrow artifact. They are included to expose the concentration of tasks: DVC supplies more than half of the instances, whereas Conan and scikit-learn supply two each. Primary estimates therefore report both micro averages and task-paired uncertainty rather than treating repository breadth as uniform.

Every task record additionally pins the start and end version, base commit, environment-setup commit, and container image name. Before dispatch, the executor resolves the image to a digest and records that digest in the trajectory. A mutable image name alone is insufficient provenance. The manifest is accepted only when it contains 48 unique instance identifiers, its repository distribution matches the source artifact, and the Arrow file’s SHA-256 matches the frozen value.

A.3 Boundary decision rubric

CONTINUE requires an incomplete admitted goal, a concrete next action, existing scope and permission, no material missing preference, and a routine or reversible engineering action. STOP requires proportionate completion evidence or absence of useful in-scope work. DEFER applies to subjective choices, credentials or missing requirements, expanded permission, destructive/external/monetary action, or uncertainty only the user can resolve. DEFER takes precedence over CONTINUE.

The final corpus will contain 60 items per label. Independent label files include a confidence level, evidence sentence, and next action or missing information. No supervisor prediction is visible to annotators. Base trajectories cannot cross development and test sets.

Candidate acquisition has its own 96-run plan: all 48 SWE-EVO tasks, two repeats, the primary worker, and the full supervisor. A literal boundary-corpus component in the deterministic run key makes every source identity disjoint from the formal matrix. A fresh boundary-scope receipt must attest capacity 96 for both logical worker and controller. The command writes trajectories

and its append-only ledger below external boundary/, never below the formal result path, and applies the USD 102/96 per-run ceiling. Every accepted source trajectory remains in the released sampling frame even when it produces no selectable item.

A.4 Annotation instrument and freeze

Annotators receive a rendered packet, not the raw result index. The packet contains the admitted goal verbatim, worker/model identity hidden behind a neutral code, the terminal provider response, the preceding tool/test evidence needed to interpret it, a repository-state summary, current permission constraints, and the boundary identifier. Costs, downstream outcomes, supervisor outputs, other labels, and reference patches are withheld.

For every item, the annotator records: label; high, medium, or low confidence; one evidence sentence; a concrete next action for CONTINUE; or the missing decision/information for DEFER. A STOP label must point to completion evidence or explain why no useful in-scope action remains. Blank rationales fail schema validation.

Two label files are sealed by hash before agreement is calculated. Cohen’s kappa is computed over the three nominal labels, without confidence weighting. If $\kappa < 0.75$, reviewers may clarify the written rubric using development cases, then independently re-label affected material once. They may not inspect frozen test predictions while revising the rubric. The released artifact preserves pre-adjudication files, their hashes, the agreement report, the adjudicated labels, and a machine-readable change log.

The exact 54/126 split is group-constrained by base trajectory. The split routine uses a frozen seed and rejects any assignment that misses a class target or places one base trajectory in both pools. Development labels can be used for prompt and regex iteration. Test labels are opened only after controller prompts, parsing, fallback, and ablation ordering are frozen.

A.5 Fault invariants

A.6 Recovery procedure

Fault injection operates at named storage boundaries rather than arbitrary wall-clock delays. For the first crash window, a failpoint terminates the process after the decided event commits and before input admission begins. The restarted drain reloads the event, re-computes no decision, and reconciles the stored input ID. For the second window, termination occurs after admission commits but before the scheduler can promote the input. Restart must discover the pending row; creating a replacement is a failure even if the transcript text would match.

Duplicate wake testing issues multiple advisory wakes for the same Session while one drain is active. The process-local coordinator must join or coalesce them, while different Session IDs remain eligible for concurrent drains. User-preemption testing inserts both steering and queued user input around an automatic pending row and verifies real-user priority plus a fresh automatic chain. Timeout, provider-error, empty-response, unavailable-model, and invalid-JSON tests record the supervisor failure mode and require explicit DEFER. Provider-error tests also distinguish retryable from non-retryable worker errors and assert the five-attempt bound. The maximum-count scenario restarts between continuations to ensure the durable count, rather than process memory, enforces the limit.

Table 5: Frozen SWE-EVO task composition. FTP and PTP denote total failing-to-passing and passing-to-passing test counts in the manifest.

Repository	Tasks	FTP	PTP	Upstream PRs
conan-io/conan	2	80	966	62
dask/dask	8	3,004	26,179	9
iterative/dvc	26	502	2,039	172
modin-project/modin	3	73	1,501	12
psf/requests	4	9	515	23
pydantic/pydantic	3	238	6,112	109
scikit-learn/scikit-learn	2	2	731	2
Total	48	3,908	38,043	389

Table 6: Frozen failure injection and required invariant.

Scenario	Required invariant
Decision before admission	Recovery admits the deterministic input exactly once.
Admission before promotion	The pending input survives restart and promotes once.
Duplicate wake	No duplicate provider execution or SessionInput row.
User preemption	Real steer/queue input takes priority and begins a new chain.
Supervisor timeout	The call terminates at 15 seconds, persists DEFER, and admits no continuation.
Invalid JSON	Parsing persists DEFER without heuristic guessing or implicit continuation.
Retryable provider failure	Retries are finite and visible in the trace.
Non-retryable provider failure	Execution stops immediately and counts as failure.
Maximum count	No input numbered <code>maxRuns + 1</code> is admitted.

Recovery success requires all invariants simultaneously: one source input ID, correct delivery mode, unchanged prompt, correct chain and sequence metadata, no skipped user input, no extra provider turn, and a traceable terminal classification. Merely reaching an idle Session is not recovery evidence.

A.7 Artifact schema

Each trajectory index row contains four groups of fields:

- *identity*: run, task, policy, worker/controller model, repeat, and attempt;
- *outcome*: timestamps, status, failure class, resolution, Fix Rate, and first-boundary prefix outcome;
- *operation*: continuation, manual, redundancy, token, cost, latency, safety, and recovery measures; and
- *provenance*: resolved model version, normalized request SHA-256, image digest, task base and implementation commits, and content-addressed trace path.

A successful record cannot carry a failure class, and a failed record must carry one. A formal run ID must belong to the 384-row matrix; a boundary-source ID must belong to the separate 96-row plan. Either must match its task, worker, controller, strategy, and repeat. The request hash is computed after defaults are normalized but before credentials are attached. Trace hashes are checked before analysis so that replacing a raw trace invalidates the index.

The cost ledger is append-only. The validator rejects unknown run IDs, duplicate task matrix identities, category or total overrun,

missing provenance, secret-shaped content, and analysis over an empty result set.

A.8 Budget reservation and failure accounting

The USD 800 cap is enforced before dispatch, not audited only after completion. The harness calculates current committed spend plus outstanding reservations. A primary run reserves at most USD 1.25, a cross-model run at most USD 3.00, and a boundary-source run at most USD 102/96; dispatch is denied if either the category or total cap would be crossed. Actual provider usage replaces the reservation when the record is appended. A mismatch between result index and ledger pauses subsequent execution for reconciliation.

Only a predefined, zero-charge infrastructure exit can consume the single rerun allowance. Examples include container creation failure before a request, corrupted task-image transfer before startup, or host interruption before provider dispatch. Model timeout, context exhaustion, controller loop, rate-limited provider attempts that incurred usage, verifier failure after execution, and budget exhaustion remain failed outcomes. This rule prevents selectively rerunning difficult or unfavorable trajectories.

The USD 50 pilot pool is gated separately. Its frozen paid task is SWE-bench Verified `psf__requests-1142`, outside the 48 SWE-EVO instances. The manifest pins dataset revision `c104f840`, Parquet and task-input SHA-256 values, the official harness commit current at image creation, the `pytest -rA` test command, and AMD64 image digest prefix `sha256:66a8b693`. Its purpose is to

validate container isolation, trace completeness, model identity capture, grader invocation, secret rejection, and cost reconciliation without selecting on a primary-benchmark outcome. Pilot output is isolated below pilot/ and never enters RQ estimates. The accepted v1.14 r9 run was unresolved (F2P 0/1, P2P 5/5, Fix Rate zero), used 17,788 prompt and 2,226 completion tokens, and cost USD 0.121062. A valid controller continuation arrived after the deadline, so the Session persisted DEFER and performed no automatic continuation. Its image also exposed 65 pre-existing build paths; a later zero-network schema-v4 replay preserved them as baseline evidence and produced an empty model-relative patch.

An engineering preflight on 2026-08-30 deviated from the intended non-primary pilot by using the smallest locally feasible official image, task `iterative__dvc_0.30.0_0.30.1`, under a host disk constraint. No normal handoff boundary or code change was obtained, and the traces were rejected from the result index. The preflight exposed an invalid final-step model-role construction and a 20-request free-tier quota. Main dispatch therefore remains disabled. If the study proceeds, the unchanged all-48 estimate is accompanied by a sensitivity analysis excluding this canary-contaminated task.

After transport, request-routing, output-limit, controller-release, and grader amendments, the v1.13 D-Robotics canary completed one accepted trajectory for each of oracle, blind, regex, and supervisor policies on `iterative__dvc_2.21.1_2.21.2`. All four first boundaries were unresolved; oracle and blind eventually resolved, regex stopped immediately, and supervisor resumed once before a 15-second controller timeout invoked the historical v1.13 regex fallback. The four rows are retained only as a single-task, unpaired paid canary and do not enter RQ estimates. Before any formal row was accepted, v1.14 changed supervisor timeout, unavailable-model, provider-error, empty-response, and malformed-output handling to DEFER, retained the 15-second bound and model matrix, and regenerated all 384 run IDs. Formal dispatch remains disabled until the 96-run source collection, annotation freeze, and fresh full-capacity v1.14 receipt are accepted. The unchanged all-48 estimate will also be accompanied by the preregistered canary-contamination sensitivity analysis.

A.9 Model and continuation request contract

Table 7 lists the provider-independent request fields. Model aliases are planning identifiers only. At runtime the executor must capture the provider’s resolved version and a SHA-256 over the normalized outbound request. A completed record with an unresolved version, missing request hash, nonzero temperature, or unrecognized default is rejected rather than treated as comparable.

All policies share the same worker system prompt, repository tool registry, permission set, per-segment step budget, context construction, and final verifier. Strategy-specific text appears only in the controller decision path and the common static continuation input. The supervisor has no repository tools and cannot mutate the task container directly. This prevents controller capability from becoming an unreported second coding agent.

The normalized request removes credentials and transport-only headers but retains provider, endpoint family, resolved model, messages, tools and schemas, sampling settings, token bounds, stop conditions, and provider feature flags. Array ordering is preserved.

JSON object keys are canonicalized before hashing. If a provider rejects a frozen field or silently substitutes a model, execution pauses and the protocol records a deviation; the harness does not rewrite earlier hashes.

Automatic continuation uses queue delivery because a terminal worker response is not itself permission to interrupt an active tool boundary. The next prompt is deterministic for oracle and blind policies. Regex and supervisor may provide a concrete safe next prompt, but the stored decision and Session input must contain the same text. A genuine user steer or queue input wins scheduling priority and resets the automatic chain, even when it is semantically equivalent to the pending automatic prompt.

A.10 System acceptance beyond model outcomes

The implementation has three independent acceptance layers. First, unit and integration tests establish controller parsing, safety precedence, configuration precedence, persistence, reconciliation, scheduling, retry limits, and V1 rejection. Second, browser tests establish that new and adopted Sessions load their server-backed state, the status indicator follows decided events, settings and slash command update the same endpoint, and a legacy server cannot display a false success. Third, fault runs validate behavior across real process restarts and failpoints.

These layers must not be substituted for one another. A passing unit test does not estimate SWE-EVO utility; a successful task trajectory does not prove crash recovery; and a rendered UI toggle does not prove runtime state changed. The artifact checklist therefore keeps system, boundary, end-to-end, visual, and publication gates separate. The paper marks mechanism claims as test-backed only when the corresponding repository evidence exists, and leaves empirical result macros pending until the appropriate frozen data gate closes.

The public API is also versioned behavior. V2 accepts a Session-level update and returns effective state; Session information projects that state to clients. V1 returns an explicit unsupported response. Configuration resolution is Session override, project configuration, then built-in default. The legacy boolean supervisor option maps into the new policy vocabulary, while invalid continuation limits outside 1–20 fail validation. None of these compatibility paths may silently enable AutoDrive for an existing Session.

B Implementation Verification Map

Evidence map. Core decision tests cover tri-state parsing and safety priority; state/config tests cover precedence and legacy mapping; controller/projector integration covers persistence and exact admission; input promotion covers source metadata; Session runner tests cover user priority and concurrent resume; Server tests cover V2 and V1 behavior; Chromium tests cover controls; and the independent evaluation package covers matrices, statistics, budget, and provenance.

Table 7: Frozen request envelope. Provider-specific defaults must be resolved and hashed by the executor.

Component	Field	Frozen value
Worker	Models	D-Robotics DeepSeek V4 Pro; Qwen 3.7 Max; DeepSeek V4 Flash
Worker	Sampling	Temperature 0; low reasoning; 4,096 output tokens; tool choice auto
Worker	Horizon	Six steps per segment; five continuations; 2,700 seconds
Supervisor	Model and sampling	D-Robotics Qwen 3.8 Max; temperature 0; no tools
Supervisor	Bound	1,024 output tokens; 15 seconds; DEFER on failure
Continuation	Delivery	Queue; "Please proceed with the next step."
Off policy	Derivation	First-boundary prefix of the same paid trajectory

C Analysis and Release Procedure

C.1 Frozen snapshot construction

Analysis begins only when the validator sees exactly 384 unique accepted run IDs, no unknown matrix identity, all trace hashes present, and category spend within the cap. The run index, ledger, trace-hash list, label files, adjudication log, environment lock, model request manifest, and analysis commit are hashed into one snapshot manifest. Derived files are written into a new directory named by that snapshot hash; analysis never updates raw JSONL in place.

The pipeline emits a run-level CSV, task-level paired CSV, policy summary JSON, boundary confusion matrices, bootstrap draws, raw and Holm-adjusted tests, cost ledger summary, failure taxonomy, and LaTeX result macros. Manuscript tables reference those macros or generated table fragments. A visible pending macro is retained unless every dependency for that result passes its gate. Thus a complete trajectory matrix cannot accidentally populate boundary metrics when labels remain unfrozen, and fault unit tests cannot populate the end-to-end recovery estimate.

C.2 Numerical audit

An independent audit recomputes counts and means from the raw index, verifies that all rates have the documented denominator, and checks that off results are prefixes of their own paid trajectories. It compares each manuscript number with the generated source, recomputes exact tests using a second implementation, and spot-checks bootstrap pairing by task ID. Values are rounded only for presentation; statistical procedures consume unrounded data.

The audit also checks direction labels. A positive supervisor delta always means supervisor minus comparator; a manual-prompt reduction is reported with an explicit sign and unit; latency uses wall-clock milliseconds; and cost uses recorded USD rather than token-derived estimates when provider invoices disagree. Null, negative, and inconclusive results remain in the main table.

C.3 Anonymization and source packaging

The anonymous and signed manuscripts share the same body, bibliography, generated result files, and appendix. A single wrapper controls author metadata. Before public release, trace sanitization removes host paths, account identifiers, and unrelated repository secrets while retaining commands, outputs, model content, and hashes needed for scientific review. Redaction locations are logged; redaction does not change numerical inputs.

The arXiv source archive contains the signed wrapper, shared source, bibliography, appendix, and generated result fragments. It excludes raw traces, credentials, build caches, PDFs, and local absolute paths. The archive is unpacked into a clean directory and compiled with the same digest-locked, network-disabled TeX image. Author names, affiliations, email, order, license, disclosure text, and final PDF require separate user approval before any upload.

D Result Publication Rules

All outcome macros are generated after the trajectory index contains 384 accepted run IDs and boundary labels pass the freeze gate. Tables and figures are regenerated from that snapshot. Hand-edited numerical results are prohibited. The final manuscript records the analysis commit, result snapshot hash, container digest, complete model identifiers, actual spend, and all deviations. The anonymous and signed PDFs are compiled from the same source with only author metadata changed.

E Full Frozen Task Manifest

Table 8 is generated directly from the digest-pinned protocol manifest. It makes the exact benchmark scope auditable without relying on a mutable dataset viewer.

Table 8: Frozen 48-task manifest. FTP and PTP are the task-level failing-to-passing and passing-to-passing test counts; PR is the upstream pull-request count.

Repository	Version evolution	FTP	PTP	PR
conan-io/conan	2.0.14–2.0.15	72	649	40
conan-io/conan	2.0.2–2.0.3	8	317	22
dask/dask	2022.9.2–2022.10.0	44	2,861	9
dask/dask	2023.3.2–2023.4.0	61	6,246	0
dask/dask	2023.6.0–2023.6.1	105	3,415	0
dask/dask	2023.6.1–2023.7.0	5	707	0
dask/dask	2023.8.0–2023.8.1	11	2,796	0
dask/dask	2023.9.2–2023.9.3	2	1,629	0
dask/dask	2024.1.0–2024.1.1	2,774	5,778	0
dask/dask	2024.3.1–2024.4.0	2	2,747	0
iterative/dvc	0.30.0–0.30.1	1	12	0
iterative/dvc	0.33.1–0.34.0	1	77	6
iterative/dvc	0.35.3–0.35.4	2	26	1
iterative/dvc	0.52.1–0.53.1	132	0	12
iterative/dvc	0.89.0–0.90.0	27	282	6
iterative/dvc	0.91.2–0.91.3	2	17	3
iterative/dvc	0.92.0–0.92.1	4	6	6
iterative/dvc	1.0.0a1–1.0.0a2	68	242	36
iterative/dvc	1.0.0b6–1.0.0	2	2	2
iterative/dvc	1.0.1–1.0.2	2	19	7
iterative/dvc	1.1.0–1.1.1	6	28	1
iterative/dvc	1.1.7–1.1.8	8	82	4
iterative/dvc	1.10.2–1.11.0	6	150	11
iterative/dvc	1.11.12–1.11.13	2	4	1
iterative/dvc	1.6.3–1.6.4	9	2	2
iterative/dvc	2.19.0–2.20.0	14	66	3
iterative/dvc	2.21.1–2.21.2	1	8	1
iterative/dvc	2.5.0–2.5.1	5	24	9
iterative/dvc	2.58.1–2.58.2	2	20	2
iterative/dvc	2.7.2–2.7.3	4	34	9
iterative/dvc	2.8.1–2.8.2	133	662	30
iterative/dvc	3.12.0–3.13.0	33	44	4
iterative/dvc	3.13.3–3.14.0	13	36	3
iterative/dvc	3.15.0–3.15.1	1	66	1
iterative/dvc	3.4.0–3.5.0	2	26	10
iterative/dvc	3.43.1–3.44.0	22	104	2
modin-project/modin	0.24.0–0.24.1	1	171	4
modin-project/modin	0.25.0–0.25.1	68	447	3
modin-project/modin	0.27.0–0.27.1	4	883	5
psf/requests	v2.12.2–v2.12.3	4	109	0
psf/requests	v2.27.0–v2.27.1	2	185	2
psf/requests	v2.4.0–v2.4.1	2	136	15
psf/requests	v2.9.0–v2.9.1	1	85	6
pydantic/pydantic	v2.6.0b1–v2.6.0	1	51	81
pydantic/pydantic	v2.7.0–v2.7.1	234	1,477	20
pydantic/pydantic	v2.7.1–v2.7.2	3	4,584	8
scikit-learn/scikit-learn	0.20.1–0.20.2	1	438	0
scikit-learn/scikit-learn	0.21.1–0.21.2	1	293	2